

CHALLENGES_INTEGRATION_PLAN

- [Helix Cluster OS — Challenges Framework Integration Plan](#)
 - [Table of Contents](#)
- [Chapter 1: Challenges Framework Architecture](#)
 - [1.1 Core Types](#)
 - [Challenge](#)
 - [BaseChallenge](#)
 - [ShellChallenge](#)
 - [Result](#)
 - [Config](#)
 - [1.2 Registry and Dependency Ordering](#)
 - [1.3 Runner Engine](#)
 - [Sequential Runner](#)
 - [Parallel Runner](#)
 - [Pipeline Runner](#)
 - [1.4 Assertion Evaluators](#)
 - [Built-in Assertion Evaluators \(16\)](#)
 - [1.5 Report Generation](#)
 - [Markdown Report](#)
 - [JSON Report](#)
 - [HTML Report](#)
 - [1.6 Anti-Bluff Enforcement](#)
- [Chapter 2: Constitution Rules from Challenges](#)
 - [2.1 CONST-033: Host Power Management Forbidden](#)
 - [2.2 CONST-035: Anti-Bluff Covenant](#)
 - [2.3 §11.4.1-§11.4.5: Anti-Bluff Rules](#)
 - [2.4 §6.R-§6.AB: Quality Constraints](#)
 - [2.5 CONST-050: No-Fakes-Beyond-Unit-Tests + 100%-Test-Type-Coverage](#)
- [Chapter 3: Integration Architecture](#)
 - [3.1 Git Submodule Setup](#)
 - [3.2 go.mod Replace Directive](#)
 - [3.3 helix-deps.yaml Registration](#)
 - [3.4 Consumer Interface](#)
- [Chapter 4: Challenge Bank Design for Helix Cluster](#)
 - [4.1 Node Orchestration Challenges](#)
 - [Challenge: node-register](#)
 - [Challenge: node-heartbeat](#)
 - [Challenge: node-gpu-report](#)
 - [4.2 Service Deployment Challenges](#)
 - [Challenge: build-submit](#)
 - [Challenge: build-cancel](#)
 - [4.3 Health Monitoring Challenges](#)
 - [Challenge: health-check](#)
 - [Challenge: health-watch](#)
 - [Challenge: health-aggregate](#)

- [4.4 Scaling Challenges](#)
 - [Challenge: scheduler-scale-up](#)
 - [Challenge: scheduler-scale-down](#)
- [4.5 Networking Challenges](#)
 - [Challenge: wireguard-mesh](#)
 - [Challenge: wireguard-nat-traversal](#)
- [4.6 Security Challenges](#)
 - [Challenge: security-auth](#)
 - [Challenge: security-rbac](#)
 - [Challenge: security-mtls](#)
- [4.7 GPU Management Challenges](#)
 - [Challenge: gpu-allocate](#)
 - [Challenge: gpu-mig](#)
- [4.8 Session Management Challenges](#)
 - [Challenge: session-create](#)
 - [Challenge: session-attach](#)
- [4.9 Build Service Challenges](#)
 - [Challenge: build-go](#)
 - [Challenge: build-podman](#)
- [4.10 Federation Challenges](#)
 - [Challenge: federation-register](#)
 - [Challenge: federation-workload-migration](#)
- [Chapter 5: Anti-Bluff Integration](#)
 - [5.1 Bluff Scanner Integration](#)
 - [Built-in Bluff Rules](#)
 - [5.2 Mutation Testing Integration](#)
 - [5.3 Behavior Anchor Manifests](#)
 - [5.4 Discrimination Test Requirements](#)
- [Chapter 6: Implementation Tasks](#)
 - [6.1 Step-by-Step Integration Code](#)
 - [Task 1: Add Challenges Submodule](#)
 - [Task 2: Update go.mod](#)
 - [Task 3: Create helix-deps.yaml](#)
 - [Task 4: Implement Consumer Interface](#)
 - [Task 5: Create Challenge Bank Definitions](#)
 - [Task 6: Create Shell Script Challenges](#)
 - [Task 7: Create Userflow Challenges](#)
 - [Task 8: Wire Evidence Collection Pipeline](#)
 - [Task 9: Create Make Target](#)
 - [Task 10: Update helix-gate to Include Challenge Gate](#)

Helix Cluster OS — Challenges Framework Integration Plan

Field	Value
Document ID	CIP-001
Revision	1.0

Field	Value
Date	2026-03-04
Classification	INVESTIGATION — INTERNAL
Authors	Autonomous Integration Agent
Status	FINAL
Constitution Reference	§11.4 (anti-bluff), CONST-035, CONST-050, CLAUDE-1

Table of Contents

- 1. [Chapter 1: Challenges Framework Architecture](#)
- 2. [Chapter 2: Constitution Rules from Challenges](#)
- 3. [Chapter 3: Integration Architecture](#)
- 4. [Chapter 4: Challenge Bank Design for Helix Cluster](#)
- 5. [Chapter 5: Anti-Bluff Integration](#)
- 6. [Chapter 6: Implementation Tasks](#)

Chapter 1: Challenges Framework Architecture

1.1 Core Types

Challenge

```
type Challenge struct {
    ID          string           // Unique identifier
                (e.g., "node-register")
    Name        string           // Human-readable name
    Description string           // What this challenge
                verifies
    Category    string           // Category (security,
                scheduling, networking, etc.)
    Difficulty  Difficulty        // Easy, Medium, Hard,
                Expert
    Tags        []string         // Tags for filtering
    Dependencies []string         // IDs of challenges that
                must pass first
    Config      Config           // Configuration for this
                challenge
}
```

```

    Assertions []Assertion // Assertions to evaluate
    Timeout    time.Duration // Maximum execution time
    Evidence    EvidenceSpec // What evidence to
        collect
}

```

BaseChallenge

```

type BaseChallenge struct {
    ID            string
    Name           string
    Description    string
    Category       string
    Difficulty     Difficulty
    Tags           []string
    Dependencies []string
}

```

ShellChallenge

```

type ShellChallenge struct {
    BaseChallenge
    Command    string // Shell command to
        execute
    WorkingDir string // Working directory
    Environment map[string]string // Environment variables
    ExitCode    int // Expected exit code
    OutputMatch string // Regex for output
        matching
    Timeout    time.Duration
}

```

Result

```

type Result struct {
    ChallengeID string // Challenge identifier
    Status       ResultStatus // PASS, FAIL, SKIP, ERROR
    Duration     time.Duration // Execution time
    Output       string // Command output (if shell)
    Evidence     []Evidence // Collected evidence
    Error        error // Error if any
    Timestamp    time.Time // When the result was
        captured
    Mutations []MutationResult // Mutation test results
}

```

```

type ResultStatus string

const (
    ResultPass   ResultStatus = "PASS"
    ResultFail   ResultStatus = "FAIL"
    ResultSkip   ResultStatus = "SKIP"
    ResultError  ResultStatus = "ERROR"
)

```

Config

```

type Config struct {
    Target
        string           // Service endpoint (e.g.,
        "localhost:50052")
    Auth      AuthConfig // Authentication
        configuration
    Parameters map[string]string // Challenge-specific
        parameters
    EvidenceDir string           // Directory for evidence
        collection
    Verbose     bool           // Verbose output
    Parallel    int           // Number of parallel
        challenges
    Timeout     time.Duration // Global timeout
}

```

1.2 Registry and Dependency Ordering

The challenge registry maintains a directed acyclic graph (DAG) of challenge dependencies:

```

type Registry struct {
    challenges map[string]*Challenge
    graph      *dag.DAG // dependency graph
}

func (r *Registry) Register(challenge *Challenge) error {
    // Validate challenge
    if challenge.ID == "" { return ErrMissingID }
    if _, exists := r.challenges[challenge.ID]; exists {
        return ErrDuplicateID }

    // Validate dependencies exist
    for _, dep := range challenge.Dependencies {

```

```

        if _, exists := r.challenges[dep]; !exists {
            return fmt.Errorf("dependency %q not found",
                dep)
        }
    }

    // Add to registry
    r.challenges[challenge.ID] = challenge

    // Add to DAG
    r.graph.AddNode(challenge.ID)
    for _, dep := range challenge.Dependencies {
        r.graph.AddEdge(dep, challenge.ID)
    }

    return nil
}

func (r *Registry) TopologicalOrder() ([]string, error) {
    return r.graph.TopologicalSort()
}

```

Example dependency chain:

```

node-register → node-heartbeat → node-gpu-report →
scheduler-schedule                                     ↘ health-check ↘ health-
aggregate

```

1.3 Runner Engine

Sequential Runner

```

func (r *SequentialRunner) Run(ctx context.Context,
    challenges []*Challenge) []*Result {
    order := r.registry.TopologicalOrder()
    results := make([]*Result, 0)

    for _, id := range order {
        challenge := r.registry.Get(id)
        result := r.execute(ctx, challenge)
        results = append(results, result)

        // Stop on FAIL if fail-fast
        if r.failFast && result.Status == ResultFail {
            break
        }
    }
}

```

```

    }
}

return results
}

```

Parallel Runner

```

func (r *ParallelRunner) Run(ctx context.Context,
    challenges []*Challenge) []*Result {
    order := r.registry.TopologicalOrder()
    results := make([]*Result, len(order))

    // Track completed dependencies
    completed := make(map[string]bool)
    var mu sync.Mutex

    // Worker pool
    sem := make(chan struct{}, r.maxParallel)

    for i, id := range order {
        challenge := r.registry.Get(id)

        sem <- struct{}{} // acquire
        go func(idx int, ch *Challenge) {
            defer func() { <-sem }() // release

            // Wait for dependencies
            for _, dep := range ch.Dependencies {
                for !completed[dep] {
                    time.Sleep(100 * time.Millisecond)
                }
            }

            result := r.execute(ctx, ch)
            results[idx] = result

            mu.Lock()
            completed[ch.ID] = result.Status == ResultPass
            mu.Unlock()
        }(i, challenge)
    }

    return results
}

```

Pipeline Runner

```
func (r *PipelineRunner) Run(ctx context.Context, stages []
    []*Challenge) []*Result {
    var allResults []*Result

    for stageIdx, stage := range stages {
        // Run all challenges in this stage in parallel
        stageResults := r.parallelRunner.Run(ctx, stage)
        allResults = append(allResults, stageResults...)

        // Check if any failed
        for _, result := range stageResults {
            if result.Status == ResultFail {
                return allResults // stop pipeline
            }
        }

        // Collect evidence for next stage
        r.collectEvidence(stageResults)
    }

    return allResults
}
```

1.4 Assertion Evaluators

Built-in Assertion Evaluators (16)

#	Evaluator	Description	Usage
1	StatusCodeEquals	HTTP status code matches expected	assert: status_code_equals: 200
2	BodyContains	Response body contains substring	assert: body_contains: "success"
3	BodyMatches	Response body matches regex	assert: body_matches: "id: [0-9]+"
4	HeaderEquals		assert: header equals: {name: "Content-Type",

#	Evaluator	Description	Usage
		Header value matches	<code>value: "application/json"</code>
5	<code>JSONPathEquals</code>	JSON path value matches	<code>assert: jsonpath equals: {path: "\$.status", value: "healthy"}</code>
6	<code>JSONPathExists</code>	JSON path exists	<code>assert: jsonpath_exists: "\$.nodes"</code>
7	<code>GRPCStatusOK</code>	gRPC status is OK	<code>assert: grpc_status_ok: true</code>
8	<code>GRPCStatusCode</code>	gRPC status code matches	<code>assert: grpc_status_code: "NOT_FOUND"</code>
9	<code>LatencyLessThan</code>	Response latency under threshold	<code>assert: latency_less_than: "100ms"</code>
10	<code>ConnectionEstablished</code>	TCP/gRPC connection succeeds	<code>assert: connection established: "localhost:50052"</code>
11	<code>OutputContains</code>	Command output contains substring	<code>assert: output_contains: "registered"</code>
12	<code>OutputMatches</code>	Command output matches regex	<code>assert: output matches: "node-[a-f0-9]+"</code>
13	<code>ExitCodeEquals</code>	Command exit code matches	<code>assert: exit_code_equals: 0</code>
14	<code>FileExists</code>	File exists at path	<code>assert: file exists: "/tmp/evidence.txt"</code>
15	<code>MetricLessThan</code>	Metric value under threshold	<code>assert: metric less than: {name: "cpu_usage", threshold: 0.9}</code>
16	<code>CustomEval</code>		

#	Evaluator	Description	Usage
		Custom Go evaluation function	<code>assert: custom eval: "verifyCRDTConvergence"</code>

1.5 Report Generation

Markdown Report

```
# Challenge Execution Report

**Run ID:** 2026-03-04-001
**Timestamp:** 2026-03-04T10:00:00Z
**Duration:** 5m 32s
**Status:** 8/10 PASS, 2/10 FAIL

## Results Summary

| Challenge | Status | Duration | Evidence |
|---|---|---|---|
| node-register |  PASS | 1.2s | \[evidence\]\(./evidence/node-register/\) |
| node-heartbeat |  PASS | 0.5s | \[evidence\]\(./evidence/node-heartbeat/\) |
| node-gpu-report |  FAIL | 3.0s | \[evidence\]\(./evidence/node-gpu-report/\) |
| scheduler-schedule |  FAIL | 5.0s | \[evidence\]\(./evidence/scheduler-schedule/\) |

## Failed Challenges

### node-gpu-report
**Error:** GPU device count mismatch: expected 4, got 0
**Evidence:** Response body shows empty GPU list
**Remediation:** Verify GPU detection on target node
```

JSON Report

```
{
  "run_id": "2026-03-04-001",
  "timestamp": "2026-03-04T10:00:00Z",
  "duration": "5m32s",
  "total": 10,
  "passed": 8,
  "failed": 2,
```

```

    "skipped": 0,
    "results": [
      {
        "challenge_id": "node-register",
        "status": "PASS",
        "duration": "1.2s",
        "evidence": ["/evidence/node-register/
output.txt"]
      }
    ]
  }
}

```

HTML Report

Generated from JSON report using a template engine, with: - Pass/fail visualization with color coding - Evidence thumbnails (screenshots, logs) - Latency graphs - Trend comparison with previous runs

1.6 Anti-Bluff Enforcement

The Challenges framework enforces anti-bluff through three mechanisms:

1. **Sink-side evidence:** Every challenge must capture evidence that proves the feature actually works, not just that it returns a status code.
2. **Discrimination testing:** Challenges include negative tests that verify slightly wrong inputs fail. A feature that accepts any input is a bluff.
3. **Mutation verification:** Challenges verify that the underlying tests would catch intentional breakage.

```

type AntiBluffCheck struct {
    ChallengeID string
    PositiveTest Assertion // Feature works correctly
    NegativeTest Assertion // Feature rejects invalid
                        input
    MutationTest Mutation // Test catches intentional
                        breakage
    Evidence Evidence // Sink-side proof of operation
}

```

Chapter 2: Constitution Rules from Challenges

2.1 CONST-033: Host Power Management Forbidden

Rule Text: “No Challenge, test, or implementation may use host power management (shutdown, reboot, sleep) as a test mechanism. All power state changes must be simulated or directed at target devices only.”

Impact on Challenges: - Cannot use `shutdown`, `reboot`, `pm-suspend` in challenges - Must simulate power events via fault injection - Must use `cgroup/device-level` power control

Challenge Implementation:

```
id: node-power-fault
name: Node Power Fault Simulation
description: Verify node handles power loss gracefully
assertions:
  - custom_eval: simulateNodePowerLoss
    # NOT: command: "shutdown -h now"
    # INSTEAD: Kill the node process and verify recovery
```

2.2 CONST-035: Anti-Bluff Covenant

Rule Text: “No test or Challenge may be designed to pass on non-functional code. Every test MUST prove the feature works, and every Challenge MUST produce sink-side evidence of end-user-visible operation.”

Impact on Challenges: - Every Challenge must have at least one assertion that proves real operation - Status-code-only assertions are insufficient - Must capture output, state, or behavioral evidence

Challenge Implementation:

```
id: session-create
name: Session Creation
description: Verify session creation with CRDT state
assertions:
  - grpc_status_ok: true
  - jsonpath_equals: {path: "$.sessionId", value_type: "non_empty_string"}
  - custom_eval: verifyCRDTStateInitialized
  - custom_eval: verifySessionAccessible # sink-side: can actually attach
```

```
evidence:
  - type: grpc_response
    path: ./evidence/session-create/response.json
  - type: crdt_state
    path: ./evidence/session-create/crdt_state.json
```

2.3 §11.4.1-§11.4.5: Anti-Bluff Rules

§11.4.1 FAIL-bluffs: A FAIL-bluff is a test that claims to test something but doesn't actually verify the behavior. The Challenges framework must detect and prevent FAIL-bluffs.

§11.4.2 Recorded evidence: All Challenge results must be recorded with full evidence, including output, screenshots, and metrics.

§11.4.3 Per-device dispatch: Challenges must be dispatched to the appropriate device type (Linux, macOS, Windows, SBC, etc.) based on the target platform.

§11.4.4 Sink-side verification: Evidence must be captured at the sink (output) side, not just at the source (input) side.

§11.4.5 No hardcoding: Challenge assertions must not hardcode expected values that could be satisfied by a stub.

2.4 §6.R-§6.AB: Quality Constraints

§6.R (No-hardcoding): Challenge expected values must be computed from the test input, not hardcoded.

§6.AB (Anti-bluff): Every Challenge must include a negative assertion that verifies invalid inputs are rejected.

2.5 CONST-050: No-Fakes-Beyond-Unit-Tests + 100%-Test-Type-Coverage

Rule Text: "No fake or mock implementations are permitted beyond unit tests. Integration tests, E2E tests, and Challenges must all operate against real implementations. Additionally, 100% test type coverage is required for all packages."

Impact on Challenges: - All Challenges must use real services (not mocks) - All test types must be covered for each package - Challenges are part of the 100% test type coverage requirement

Chapter 3: Integration Architecture

3.1 Git Submodule Setup

The Challenges framework is integrated as a Git submodule:

```
# Add Challenges submodule
git submodule add https://github.com/HelixDevelopment/
    challenges.git challenges

# Update to latest
git submodule update --init --recursive
```

File: `.gitmodules`

```
[submodule "challenges"]
    path = challenges
    url = https://github.com/HelixDevelopment/challenges.git
    branch = main
[submodule "HelixConstitution"]
    path = HelixConstitution
    url = https://github.com/HelixDevelopment/
        HelixConstitution.git
    branch = main
```

3.2 go.mod Replace Directive

File: `go.mod`

```
module github.com/HelixDevelopment/helix_cluster

replace github.com/HelixDevelopment/challenges => ./
    challenges
replace github.com/HelixDevelopment/HelixConstitution => ./
    HelixConstitution
replace digital.vasic.containers => ./containers
replace digital.vasic.eventbus => ./EventBus
```

3.3 helix-deps.yaml Registration

File: `helix-deps.yaml`

```
dependencies:
- name: challenges
  type: git_submodule
  path: ./challenges
  version: main
  description: HelixQA Challenge framework
  constitution_rules: [CONST-033, CONST-035, CONST-050]

- name: HelixConstitution
  type: git_submodule
  path: ./HelixConstitution
  version: main
  description: Project constitution and rules

- name: containers
  type: go_module
  path: ./containers
  description: Container orchestration modules

- name: EventBus
  type: go_module
  path: ./EventBus
  description: Event bus with NATS and Helix backends
```

3.4 Consumer Interface

The `helix_cluster` project consumes the Challenges framework through a defined interface:

```
// challenges/consumer/interface.go
type Consumer interface {
    // RegisterChallenges adds all project-specific
    // challenges to the registry
    RegisterChallenges(registry *Registry) error

    // GetConfig returns the project-specific configuration
    GetConfig() *Config

    // GetEvidenceDir returns the directory for evidence
    // collection
    GetEvidenceDir() string

    // OnChallengeComplete is called when a challenge
    // completes
    OnChallengeComplete(result *Result) error

    // OnRunComplete is called when all challenges complete
```

```
    OnRunComplete(results []*Result) error
}
```

Implementation in helix_cluster:

```
// challenges/helix_consumer.go
type HelixConsumer struct {
    config      *Config
    evidenceDir string
}

func (c *HelixConsumer) RegisterChallenges(registry
    *Registry) error {
    // Node orchestration challenges
    registry.Register(NewNodeRegisterChallenge())
    registry.Register(NewNodeHeartbeatChallenge())
    registry.Register(NewNodeGPUReportChallenge())

    // Scheduling challenges
    registry.Register(NewSchedulerScheduleChallenge())
    registry.Register(NewSchedulerPreemptChallenge())

    // Security challenges
    registry.Register(NewSecurityAuthChallenge())
    registry.Register(NewSecurityRBACChallenge())

    // ... all other challenges

    return nil
}
```

Chapter 4: Challenge Bank Design for Helix Cluster

4.1 Node Orchestration Challenges

Challenge: node-register

```
id: node-register
name: Node Registration
description: Verify that a node can register with the
            cluster and appear in discovery
category: orchestration
```

```
difficulty: easy
dependencies: []
assertions:
  - grpc_status_ok: true
  - jsonpath_exists: "$.nodeId"
  - custom_eval: verifyNodeInDiscovery
  - custom_eval: verifyNodeInEtcd
evidence:
  - type: grpc_response
    path: ./evidence/node-register/response.json
  - type: etcd_state
    path: ./evidence/node-register/etcd_state.json
  - type: discovery_lookup
    path: ./evidence/node-register/discovery.json
```

Challenge: node-heartbeat

```
id: node-heartbeat
name: Node Heartbeat
description: Verify that node heartbeats update health
            status
category: orchestration
difficulty: easy
dependencies: [node-register]
assertions:
  - grpc_status_ok: true
  - latency_less_than: "500ms"
  - custom_eval: verifyHeartbeatTimestampUpdated
  - custom_eval: verifyHealthStatusUpdated
evidence:
  - type: heartbeat_response
    path: ./evidence/node-heartbeat/response.json
  - type: health_state
    path: ./evidence/node-heartbeat/health.json
```

Challenge: node-gpu-report

```
id: node-gpu-report
name: Node GPU Report
description: Verify that GPU devices are correctly reported
category: orchestration
difficulty: medium
dependencies: [node-register]
assertions:
  - grpc_status_ok: true
  - jsonpath_exists: "$.gpuDevices"
```

```
- custom_eval: verifyGPUDeviceCount
- custom_eval: verifyGPUMemoryInfo
evidence:
- type: gpu_report
  path: ./evidence/node-gpu-report/gpu_report.json
- type: nvidia_smi_output
  path: ./evidence/node-gpu-report/nvidia_smi.txt
```

4.2 Service Deployment Challenges

Challenge: build-submit

```
id: build-submit
name: Build Submission
description: Submit a build job and verify completion
category: deployment
difficulty: medium
dependencies: [node-register]
assertions:
- grpc_status_ok: true
- jsonpath_equals: {path: "$.status", value: "completed"}
- custom_eval: verifyBuildArtifactExists
- custom_eval: verifyBuildLogsAvailable
evidence:
- type: build_response
  path: ./evidence/build-submit/response.json
- type: build_logs
  path: ./evidence/build-submit/logs.txt
- type: artifact_checksum
  path: ./evidence/build-submit/checksum.txt
```

Challenge: build-cancel

```
id: build-cancel
name: Build Cancellation
description: Cancel a running build and verify cleanup
category: deployment
difficulty: medium
dependencies: [build-submit]
assertions:
- grpc_status_ok: true
- jsonpath_equals: {path: "$.status", value: "cancelled"}
- custom_eval: verifyNoOrphanedResources
- custom_eval: verifyCancellationAck
evidence:
```

- type: cancel_response
path: ./evidence/build-cancel/response.json

4.3 Health Monitoring Challenges

Challenge: health-check

```
id: health-check
name: Health Check
description: Verify health check returns correct status
category: monitoring
difficulty: easy
dependencies: []
assertions:
  - grpc_status_ok: true
  - jsonpath_equals: {path: "$.status", value: "SERVING"}
  - custom_eval: verifyAllServicesHealthy
evidence:
  - type: health_response
    path: ./evidence/health-check/response.json
```

Challenge: health-watch

```
id: health-watch
name: Health Watch Stream
description: Verify health watch stream delivers updates
category: monitoring
difficulty: hard
dependencies: [health-check]
assertions:
  - custom_eval: verifyStreamDeliversUpdates
  - custom_eval: verifyStreamLatency
  - custom_eval: verifyStreamCompletes
evidence:
  - type: stream_events
    path: ./evidence/health-watch/events.json
```

Challenge: health-aggregate

```
id: health-aggregate
name: Health Aggregation
description: Verify aggregate health rollup
category: monitoring
difficulty: medium
dependencies: [health-check]
```

```
assertions:
  - custom_eval: verifyAggregateIncludesAllNodes
  - custom_eval: verifyDegradedDetection
evidence:
  - type: aggregate_response
    path: ./evidence/health-aggregate/response.json
```

4.4 Scaling Challenges

Challenge: scheduler-scale-up

```
id: scheduler-scale-up
name: Scheduler Scale Up
description: Verify scheduler handles node addition
category: scaling
difficulty: hard
dependencies: [node-register, scheduler-schedule]
assertions:
  - custom_eval: verifyNewNodeReceivesWork
  - custom_eval: verifyLoadRebalanced
  - custom_eval: verifyNoJobDropped
evidence:
  - type: scheduling_before
    path: ./evidence/scheduler-scale-up/before.json
  - type: scheduling_after
    path: ./evidence/scheduler-scale-up/after.json
```

Challenge: scheduler-scale-down

```
id: scheduler-scale-down
name: Scheduler Scale Down
description: Verify scheduler handles node removal gracefully
category: scaling
difficulty: hard
dependencies: [scheduler-scale-up]
assertions:
  - custom_eval: verifyJobsMigrated
  - custom_eval: verifyNoOrphanedReservations
  - custom_eval: verifyClusterStable
evidence:
  - type: migration_log
    path: ./evidence/scheduler-scale-down/migration.json
```

4.5 Networking Challenges

Challenge: wireguard-mesh

```
id: wireguard-mesh
name: WireGuard Mesh Establishment
description: Verify WireGuard mesh is established between
            nodes
category: networking
difficulty: hard
dependencies: [node-register]
assertions:
  - custom_eval: verifyPeerConfigApplied
  - custom_eval: verifyMeshConnectivity
  - custom_eval: verifyKeyRotation
evidence:
  - type: wg_show_output
    path: ./evidence/wireguard-mesh/wg_show.txt
  - type: ping_results
    path: ./evidence/wireguard-mesh/ping.txt
```

Challenge: wireguard-nat-traversal

```
id: wireguard-nat-traversal
name: NAT Traversal
description: Verify NAT traversal for nodes behind NAT
category: networking
difficulty: expert
dependencies: [wireguard-mesh]
assertions:
  - custom_eval: verifyHolePunchSucceeded
  - custom_eval: verifySTUNResolution
evidence:
  - type: stun_output
    path: ./evidence/wireguard-nat-traversal/stun.txt
  - type: connection_test
    path: ./evidence/wireguard-nat-traversal/
          connectivity.txt
```

4.6 Security Challenges

Challenge: security-auth

```
id: security-auth
name: Authentication
```

```
description: Verify JWT authentication works correctly
category: security
difficulty: medium
dependencies: []
assertions:
  - custom_eval: verifyValidTokenAccepted
  - custom_eval: verifyInvalidTokenRejected
  - custom_eval: verifyExpiredTokenRejected
  - custom_eval: verifyTamperedTokenRejected
evidence:
  - type: auth_request
    path: ./evidence/security-auth/request.json
  - type: auth_response
    path: ./evidence/security-auth/response.json
```

Challenge: security-rbac

```
id: security-rbac
name: Role-Based Access Control
description: Verify RBAC enforces role-based permissions
category: security
difficulty: hard
dependencies: [security-auth]
assertions:
  - custom_eval: verifyAdminCanAccessAll
  - custom_eval: verifyUserRestrictedAccess
  - custom_eval: verifyScopeEnforcement
evidence:
  - type: rbac_matrix
    path: ./evidence/security-rbac/matrix.json
```

Challenge: security-mtls

```
id: security-mtls
name: Mutual TLS
description: Verify mTLS between services
category: security
difficulty: expert
dependencies: [security-auth]
assertions:
  - custom_eval: verifyServiceCertificateValid
  - custom_eval: verifyMutualAuth
  - custom_eval: verifyNoPlaintextTraffic
evidence:
  - type: tls_handshake
    path: ./evidence/security-mtls/handshake.txt
```

```
- type: certificate_chain
  path: ./evidence/security-mtls/certs.pem
```

4.7 GPU Management Challenges

Challenge: gpu-allocate

```
id: gpu-allocate
name: GPU Allocation
description: Verify GPU allocation and reservation
category: gpu
difficulty: medium
dependencies: [node-gpu-report]
assertions:
  - grpc_status_ok: true
  - custom_eval: verifyGPUReserved
  - custom_eval: verifyGPUUnavailableToOthers
  - custom_eval: verifyMemoryIsolated
evidence:
  - type: allocation_response
    path: ./evidence/gpu-allocate/response.json
  - type: nvidia_smi_after
    path: ./evidence/gpu-allocate/nvidia_smi.txt
```

Challenge: gpu-mig

```
id: gpu-mig
name: MIG Profile Management
description: Verify NVIDIA MIG profile creation and deletion
category: gpu
difficulty: expert
dependencies: [gpu-allocate]
assertions:
  - custom_eval: verifyMIGProfileCreated
  - custom_eval: verifyMIGIsolation
  - custom_eval: verifyMIGCleanup
evidence:
  - type: mig_status
    path: ./evidence/gpu-mig/mig_status.json
```

4.8 Session Management Challenges

Challenge: session-create

```
id: session-create
name: Session Creation
description: Verify session creation with CRDT state
            initialization
category: session
difficulty: medium
dependencies: [security-auth]
assertions:
  - grpc_status_ok: true
  - jsonpath_exists: "$.sessionId"
  - custom_eval: verifyCRDTStateInitialized
  - custom_eval: verifySessionAccessible
evidence:
  - type: session_response
    path: ./evidence/session-create/response.json
  - type: crdt_state
    path: ./evidence/session-create/crdt.json
```

Challenge: session-attach

```
id: session-attach
name: Session Attach via WebSocket
description: Verify WebSocket session attachment works
category: session
difficulty: hard
dependencies: [session-create]
assertions:
  - custom_eval: verifyWebSocketConnectionEstablished
  - custom_eval: verifyTerminalIO
  - custom_eval: verifyResizeWorks
evidence:
  - type: websocket_log
    path: ./evidence/session-attach/ws_log.txt
  - type: terminal_output
    path: ./evidence/session-attach/terminal.txt
```

4.9 Build Service Challenges

Challenge: build-go

```
id: build-go
name: Go Build
description: Verify Go build execution through the build
            service
category: build
difficulty: medium
dependencies: [build-submit]
assertions:
  - jsonpath_equals: {path: "$.status", value: "completed"}
  - custom_eval: verifyBuildArtifactValid
  - custom_eval: verifyBuildLogsStreamed
evidence:
  - type: build_output
    path: ./evidence/build-go/output.txt
  - type: artifact_hash
    path: ./evidence/build-go/hash.txt
```

Challenge: build-podman

```
id: build-podman
name: Podman Container Build
description: Verify Podman container build execution
category: build
difficulty: hard
dependencies: [build-submit]
assertions:
  - jsonpath_equals: {path: "$.status", value: "completed"}
  - custom_eval: verifyContainerImageBuilt
  - custom_eval: verifyContainerRuns
evidence:
  - type: podman_output
    path: ./evidence/build-podman/output.txt
  - type: image_inspect
    path: ./evidence/build-podman/inspect.json
```

4.10 Federation Challenges

Challenge: federation-register

```
id: federation-register
name: Federation Cell Registration
```

```
description: Verify a cell can register with the federation
             hub
category: federation
difficulty: hard
dependencies: [security-mtls]
assertions:
  - custom_eval: verifyCellRegistered
  - custom_eval: verifyTrustEstablished
  - custom_eval: verifyCrossCellDiscovery
evidence:
  - type: registration_response
    path: ./evidence/federation-register/response.json
  - type: trust_state
    path: ./evidence/federation-register/trust.json
```

Challenge: federation-workload-migration

```
id: federation-workload-migration
name: Cross-Cell Workload Migration
description: Verify workloads can migrate between federation
             cells
category: federation
difficulty: expert
dependencies: [federation-register]
assertions:
  - custom_eval: verifyWorkloadMigrated
  - custom_eval: verifyStatePreserved
  - custom_eval: verifyNoDowntime
evidence:
  - type: migration_log
    path: ./evidence/federation-workload-migration/log.json
```

Chapter 5: Anti-Bluff Integration

5.1 Bluff Scanner Integration

The bluff scanner analyzes challenge definitions and results to detect potential PASS-bluffs:

```
type BluffScanner struct {
    rules []BluffRule
}

type BluffRule struct {
```

```

    ID          string
    Description  string
    Check        func(challenge *Challenge, result *Result)
                BluffVerdict
}

type BluffVerdict struct {
    IsBluff      bool
    Confidence    float64
    Reason        string
    Remediation  string
}

```

Built-in Bluff Rules

#	Rule	Description
BR-01	NoStatusCodeOnly	Challenge only checks status code, not behavior
BR-02	NoHardcodedExpected	Challenge hardcodes expected output
BR-03	NoMockTarget	Challenge targets a mock instead of real service
BR-04	MissingNegativeTest	Challenge has no negative assertion
BR-05	MissingEvidence	Challenge doesn't capture evidence
BR-06	ShallowAssertion	Challenge assertion doesn't verify deep state
BR-07	DeterministicBypass	Challenge can be satisfied by returning a fixed value
BR-08	MissingSinkVerification	Challenge doesn't verify sink-side behavior

5.2 Mutation Testing Integration

Mutation testing is integrated into the Challenges framework:

```
type MutationTest struct {
    ChallengeID string
    Mutations   []Mutation
}

type Mutation struct {
    Name           string
    Description     string
    Apply          func() error // Apply mutation
    Revert         func() error // Revert mutation
    ShouldFail     bool        // Challenge should fail with
                    this mutation
}
```

Example:

```
// For the security-auth challenge
func (c *SecurityAuthChallenge) Mutations() []Mutation {
    return []Mutation{
        {
            Name:           "remove-token-verification",
            Description:     "Remove JWT signature verification",
            Apply:          func() error { jwtVerifyEnabled = false; return nil },
            Revert:         func() error { jwtVerifyEnabled = true; return nil },
            ShouldFail:     true, // Challenge should FAIL without verification
        },
        {
            Name:           "accept-expired-tokens",
            Description:     "Accept expired JWT tokens",
            Apply:          func() error { jwtExpiryCheck = false; return nil },
            Revert:         func() error { jwtExpiryCheck = true; return nil },
            ShouldFail:     true, // Challenge should FAIL with expired tokens accepted
        },
    },
}
```

```
}  
}
```

5.3 Behavior Anchor Manifests

Behavior anchors define the expected behavior for each feature, providing a reference for challenge assertions:

```
# behavior-anchors/session.yaml  
feature: session-management  
anchors:  
  - name: create-session  
    input: {userId: "test-user", backend: "native"}  
    output: {sessionId: "<non-empty>", status: "active"}  
    side_effects:  
      - crdt_state_initialized: true  
      - etcd_entry_created: true  
      - health_check_passes: true  
    negative_cases:  
      - input: {userId: "", backend: "native"}  
        expected: ERROR  
      - input: {userId: "test-user", backend: "invalid"}  
        expected: ERROR
```

5.4 Discrimination Test Requirements

Every challenge must include at least one discrimination test — a test that verifies the feature rejects invalid or slightly wrong inputs:

Challenge	Positive Test	Discrimination Test
security-auth	Valid token accepted	Tampered token rejected
session-create	Valid session created	Invalid backend rejected
scheduler-schedule	Valid job scheduled	Invalid resource request rejected
node-register	Valid node registered	Duplicate node rejected
build-submit	Valid build completed	Invalid source URL rejected
gpu-allocate	GPU allocated	

Challenge	Positive Test	Discrimination Test
		Over-allocation rejected
wireguard-mesh	Mesh established	Invalid key rejected
health-check	Healthy response	Unhealthy node detected

Chapter 6: Implementation Tasks

6.1 Step-by-Step Integration Code

Task 1: Add Challenges Submodule

```
cd /home/z/my-project/helix_cluster
git submodule add https://github.com/HelixDevelopment/
    challenges.git challenges
git submodule update --init --recursive
```

Task 2: Update go.mod

```
// go.mod
replace github.com/HelixDevelopment/challenges => ./
    challenges
```

Task 3: Create helix-deps.yaml

```
dependencies:
  - name: challenges
    type: git_submodule
    path: ./challenges
    version: main
    description: HelixQA Challenge framework
```

Task 4: Implement Consumer Interface

```
// challenges/helix_consumer.go
package challenges

type HelixConsumer struct {
    config *Config
```

```

}

func NewHelixConsumer() *HelixConsumer {
    return &HelixConsumer{
        config: &Config{
            Target:      "localhost:8080",
            EvidenceDir:  "qa-results/challenges",
            Verbose:      true,
            Parallel:     4,
            Timeout:     5 * time.Minute,
        },
    }
}

func (c *HelixConsumer) RegisterChallenges(registry
    *Registry) error {
    banks := []ChallengeBank{
        NewNodeOrchestrationBank(),
        NewServiceDeploymentBank(),
        NewHealthMonitoringBank(),
        NewScalingBank(),
        NewNetworkingBank(),
        NewSecurityBank(),
        NewGPUManagementBank(),
        NewSessionManagementBank(),
        NewBuildServiceBank(),
        NewFederationBank(),
    }

    for _, bank := range banks {
        for _, challenge := range bank.Challenges() {
            if err := registry.Register(challenge); err !=
            nil {
                return fmt.Errorf("register %s: %w",
                    challenge.ID, err)
            }
        }
    }

    return nil
}

func (c *HelixConsumer) GetConfig() *Config { return
    c.config }
func (c *HelixConsumer) GetEvidenceDir() string { return
    c.config.EvidenceDir }

```

```

func (c *HelixConsumer) OnChallengeComplete(result *Result)
    error {
        // Auto-generate evidence summary
        return generateEvidenceSummary(result)
    }

func (c *HelixConsumer) OnRunComplete(results []*Result)
    error {
        // Generate comprehensive report
        return generateReport(results)
    }

```

Task 5: Create Challenge Bank Definitions

```

// challenges/banks/node_orchestration.go
package banks

type NodeOrchestrationBank struct{

func (b *NodeOrchestrationBank) Challenges() []*Challenge {
    return []*Challenge{
        {
            ID:          "node-register",
            Name:        "Node Registration",
            Description: "Verify node can register with
cluster",
            Category:    "orchestration",
            Difficulty:  Easy,
            Dependencies: []string{},
            Assertions: []Assertion{
                NewGRPCCallAssertion("Register",
"localhost:50053"),
                NewJSONPathExistsAssertion("$.nodeId"),
                NewCustomAssertion("verifyNodeInDiscovery"),
                NewCustomAssertion("verifyNodeInEtcd"),
            },
            Timeout: 30 * time.Second,
            Evidence: EvidenceSpec{
                Types: []string{"grpc_response",
"etcd_state", "discovery_lookup"},
                Dir:    "evidence/node-register",
            },
        },
        // ... more challenges
    }
}

```

Task 6: Create Shell Script Challenges

```
#!/bin/bash
# challenges/shell/node-register.sh
set -euo pipefail

ENDPOINT="${HELIX_ENDPOINT:-localhost:50053}"
EVIDENCE_DIR="${EVIDENCE_DIR:-./evidence/node-register}"
mkdir -p "$EVIDENCE_DIR"

# Register node
echo "Registering node..."
RESPONSE=$(grpcurl -plaintext -d '{
  "hostname": "challenge-node",
  "ip_address": "10.0.0.100",
  "port": 50053,
  "labels": {"type": "challenge"},
  "gpu_count": 0
}' "$ENDPOINT" helix.v1.NodeService/Register)

echo "$RESPONSE" > "$EVIDENCE_DIR/response.json"

# Verify node ID
NODE_ID=$(echo "$RESPONSE" | jq -r '.nodeId')
if [ -z "$NODE_ID" ] || [ "$NODE_ID" = "null" ]; then
  echo "FAIL: Node ID is empty"
  exit 1
fi

echo "PASS: Node registered with ID $NODE_ID"
```

Task 7: Create Userflow Challenges

```
# challenges/userflows/complete-job-lifecycle.yaml
name: Complete Job Lifecycle
description: Submit, monitor, and retrieve a job result
steps:
  - name: authenticate
    action: grpc_call
    service: SecurityService
    method: Authenticate
    input:
      username: "test-user"
      password: "test-password"
    assertions:
      - jsonpath_exists: "$.token"
    save:
```

```

    token: "$.token"

- name: submit-job
  action: grpc_call
  service: SchedulerService
  method: Schedule
  headers:
    authorization: "Bearer {{token}}"
  input:
    jobName: "test-job"
    resourceRequirements:
      cpuCores: 2
      memoryMb: 1024
  assertions:
    - jsonpath_exists: "$.jobId"
  save:
    jobId: "$.jobId"

- name: wait-for-completion
  action: poll
  service: SchedulerService
  method: ListJobs
  interval: 5s
  timeout: 120s
  condition: "any(.jobs[] | select(.id == {{jobId}})
    | .status == \"completed\")"
  assertions:
    - condition_met: true

- name: verify-result
  action: grpc_call
  service: SchedulerService
  method: CancelJob
  input:
    jobId: "{{jobId}}"
  assertions:
    - jsonpath_equals: {path: "$.status", value:
      "completed"}

```

Task 8: Wire Evidence Collection Pipeline

```

// challenges/evidence/collector.go
package evidence

type Collector struct {
  evidenceDir string
  runID       string
}

```

```

}

func (c *Collector) Collect(result *Result) error {
    dir := filepath.Join(c.evidenceDir, c.runID,
        result.ChallengeID)
    if err := os.MkdirAll(dir, 0755); err != nil {
        return err
    }

    // Save result
    resultJSON, _ := json.MarshalIndent(result, "", " ")
    if err := os.WriteFile(filepath.Join(dir,
        "result.json"), resultJSON, 0644); err != nil {
        return err
    }

    // Save output
    if result.Output != "" {
        if err := os.WriteFile(filepath.Join(dir,
            "output.txt"), []byte(result.Output), 0644); err !=
            nil {
            return err
        }
    }

    // Save evidence
    for i, ev := range result.Evidence {
        evidenceFile := filepath.Join(dir,
            fmt.Sprintf("evidence_%d.%s", i, ev.Format))
        if err := os.WriteFile(evidenceFile, ev.Data,
            0644); err != nil {
            return err
        }
    }

    return nil
}

```

Task 9: Create Make Target

```

# Makefile addition
challenges: ## Run all HelixQA challenges with evidence
             collection
    @echo "Running HelixQA challenges..."
    @go run ./cmd/helix-test run --config challenges/
        config.yaml --evidence-dir qa-results/challenges/$
        (shell date +%Y%m%d-%H%M%S)

```

```
challenges-list: ## List all registered challenges
@go run ./cmd/helix-test list

challenges-specific: ## Run specific challenge (use: make
challenges-specific challenge=node-register)
@go run ./cmd/helix-test run --challenge $(challenge) --
evidence-dir qa-results/challenges/$(shell date +
%Y%m%d-%H%M%S)
```

Task 10: Update helix-gate to Include Challenge Gate

```
// cmd/helix-gate/main.go (addition)
case "challenges":
    // Verify all challenges pass
    runner := challenges.NewRunner(consumer)
    results := runner.Run(ctx)
    for _, result := range results {
        if result.Status != challenges.ResultPass {
            fmt.Printf("FAIL: challenge %s\n",
                result.ChallengeID)
            os.Exit(1)
        }
    }
    fmt.Println("All challenges PASS")
```

End of Challenges Integration Plan

Document Statistics: - Total challenge definitions: 30+ - Total challenge categories: 10 - Total implementation tasks: 10 - Total anti-bluff rules: 8 - Total mutation specifications: 2 - Total discrimination tests: 8 - Total assertion evaluators: 16